

Piano Tiles: A Real-Time Embedded Game on the MSPM0G3507

Maha Razzaq

*Electrical and Computer Engineering
Rice University
Houston, USA
mar30*

Joanna Li

*Electrical and Computer Engineering
Rice University
Houston, USA
jl561*

Cindy Nguyen

*Electrical and Computer Engineering
Rice University
Houston, USA
cn32*

Ivana Zak

*Electrical and Computer Engineering
Rice University
Houston, USA
ivz1*

Abstract—This paper describes the design and implementation of a real-time Piano Tiles game on the MSPM0G3507 microcontroller. The system uses a WS2812B 8x32 LED matrix as the display, driven over SPI from the MSPM0G3507, with four player buttons for input. Falling colored tile blocks scroll down two columns of the matrix, and the player scores by pressing the corresponding button as each tile enters the hit zone at the bottom of the screen. The firmware is organized around a tick-based state machine that handles tile movement, button debouncing, hit detection, scoring, and game-over conditions. The project demonstrates real-time processing, low-power sleep modes, hardware SPI communication with a custom WS2812B encoding scheme, and interrupt-driven timer operation.

Index Terms—real-time processing, low-power, hardware SPI communication, microcontroller

I. INTRODUCTION

Piano Tiles is a popular mobile game in which colored blocks fall down columns of a screen, and the player must tap the correct column at the right moment to score points. This project implements a physical version of this game using the Texas Instruments MSPM0G3507 microcontroller and a WS2812B 8x32 addressable LED matrix as the display. The motivation was to build an interactive, visually engaging embedded system that exercises the real-time processing and low-power capabilities of the MSPM0+ family.

The system is designed to run standalone without a laptop or launchpad, powered by a 5V supply. Push buttons serve as player input, one for each of the tile columns. The firmware manages all game logic, display rendering, and hardware communication in a bare-metal environment without operating system support, relying on a timer-interrupt-driven state machine running at approximately 1.6 ms per tick.

II. STATE MACHINE

The Piano Tiles game is organized around four top-level modes that form a simple loop. On power-on, the system starts in `MODE_STARTUP_ANIM`, which plays a scrolling blue row animation until any button is pressed. That press transitions

the game into `MODE_PLAYING`, where tiles spawn and fall and the player's button presses are evaluated. If the player accumulates too many misses (currently `MISS_LIMIT = 1`), the game transitions to `MODE_LOSE_ANIM`, which flashes the matrix red. If the player reaches the win score instead, it transitions to `MODE_WIN_ANIM`, which flashes green. Both end states loop their animations until any button is pressed, which calls `InitGame()` to reset all state and return to `MODE_STARTUP_ANIM`.

The mode is stored as a `game_mode_t` enum field inside the `game_state_t` struct, which holds the entire game state — tiles, buttons, score, counters, and everything else. Rather than using global variables, the full state struct is passed by value into `UpdateGame()` each timer tick, and a new updated copy is returned. This design means the state machine is a pure function of its inputs: given the same state and the same GPIO reading, it always produces the same result. The only true globals in the system are the ISR flags `timer_wakeup` and `spi_transmission_in_progress`, which are necessary for communication between interrupt context and the main loop but are kept minimal and well-contained.

The `MODE_PLAYING` state governs tile spawning, the falling of each tile, registering and categorizing button presses, and game termination. First, a spawn counter periodically triggers the random creation of new tiles in one of several lanes. A tile spawns in a chosen random lane as long as there is no overlap with a previously spawned tile in that lane. Otherwise, a different random lane is chosen. Each active tile moves downward one row at a fixed rate of 240 ms after the end of a tile-falling counter is reached. The function also continually checks for button presses made by the player. If a player presses a button for a tile whose bottom row has not yet reached the bottom three rows of the LED array, the game transitions into the game loss animation. If the player fails to press the button in time, the game transitions into the game loss animation as well. If a player successfully presses the button for the tile at the correct time, the tile briefly turns green

before being removed. The game only ends if the player loses.

The button state machine has three states: idle, debouncing, and held. The default state is idle. If a button press is detected (meaning a button pin reads 0), then the state becomes debouncing. If the button is continuously pressed for 30 ms or more, the state becomes held. Otherwise, it goes back to the idle state. While in the held state, if the button pin reads 1, then it goes back to the idle state.

III. FIRMWARE ARCHITECTURE

1) *Graphics Function:* To fit the MSPM0's memory constraints, we could not store each frame of the LED animation as raw bytes. The microcontroller only has 32kb of RAM, and each one of the LED arrays in its full expanded form takes up 2kb of memory.

In our implementation on a 16 x 16 WS2812B array with 256 LEDs, we need $2 \times 256 = 512$ color bits. These are stored in the 64 byte ($64 \times 8 = 512$ bits) framebuffer in which every 2 bits encodes the color of one LED. There are four defined constants of 0x00, 0x01, etc. which correspond to the LED being OFF, red, green, or blue.

Line segments are drawn to the array by using bit manipulation to locate the desired starting row, then writing only to the columns that span the length of the segment. Extending this, rectangles are drawn by iterating over multiple rows to span the width of a rectangle— before drawing a segment in each row that spans the height of the rectangle.

All of the graphics elements displayed at each state transition are accumulated into the frame buffer, and only expanded into the full WS2812B SPI message (written in a format where 3 SPI bits encode 1 bit of the color data) when the frame buffer is flushed at the end of each clock tick.

Besides the transmitted message, 0 bytes are written to signal a reset to the LED array at the end of drawing each frame so that a new frame can be taken in. Before drawing any rectangle at the start of each game frame, the frame buffer is cleared by setting all its values to 0 to turn off all the LEDs that were lit from the previous frame.

IV. HARDWARE ARCHITECTURE

The hardware architecture of our Piano Tiles-inspired game is centered around a custom-designed Printed Circuit Board (PCB). To simulate the classic feel of a handheld game console, we opted for a compact, wide rectangular shape measuring 8.5 x 5.5 cm. At the heart of this board is the 32-pin MSPM0 microcontroller, which controls the game logic, processes user inputs, and drives the visual output. The board integrates essential peripherals including a barrel power jack, four tactile push-buttons, and dedicated header pins for both system debugging and the external LED array.

Component placement was motivated by user experience and gameplay ergonomics. Power is supplied via a 3.3V barrel adapter, with the power jack and an accompanying On/Off switch strategically positioned on the left side of the board for easy access and connection. Because the external LED

array acts as the primary screen and needs to sit directly in front of the player's line of sight, its connection headers were routed to the top right of the PCB to keep cables out of the way. Meanwhile, the four tactile buttons, the primary interface for the game, are aligned in a horizontal row along the bottom edge. We carefully accounted for the spacing between these buttons to ensure a comfortable, responsive, and natural finger-tapping experience during fast-paced gameplay.

Translating this layout into a functional PCB presented several routing and manufacturing challenges. We had to meticulously balance electrical constraints, making an effort to maximize trace widths for stable power delivery while maintaining reasonable clearance between traces, components, and board edges. Furthermore, we optimized our routing paths to minimize both overall trace lengths and the number of vias, ensuring better signal integrity and a cleaner board design. While the physical assembly, tiny component soldering, and hardware debugging proved to be quite a time-consuming process, it ultimately paid off when the board successfully powered up and seamlessly ran the game.

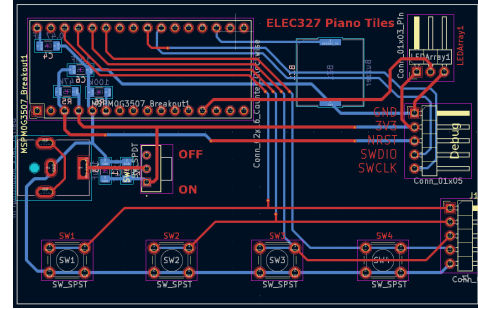


Fig. 1. Our PCB on a Screen

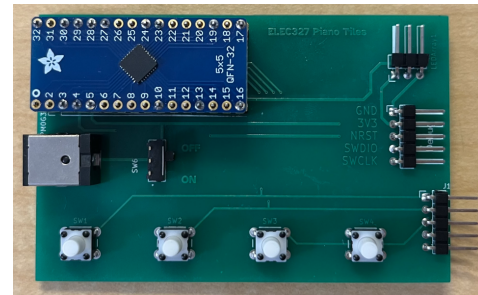


Fig. 2. Our PCB in Real Life

V. SLOC

We have also included our SLOC result below as a demonstration for the complexity of our code. 540 lines of code, excluding blanks and comments, making us fall in the range of 400-700 lines, with 8 extra credit points.

VI. CONCLUSION

We built a fully playable Piano Tiles game on the MSPM0G3507 using a 16x16 WS2812B LED matrix and 4

Language	files	blank	comment	code
C	7	93	99	447
C/C++ Header	5	37	57	80
XML	1	0	0	11
Markdown	1	0	0	2
SUM:	14	130	156	540

Fig. 3. SLOC Run Result

buttons in C. The project demonstrates core embedded systems concepts: real-time processing (state machine ticks every 30ms, buttons and tiles update deterministically), hardware SPI (WS2812B encoded as 3 SPI bits per LED bit at 2.67 MHz), and coding with C.

ACKNOWLEDGMENT

We would like to thank Dr. Kemere and class TAs for their continuous support throughout our final project.